TECHNICAL WHITE PAPER · VOL. I · DIGITAL EDITION · 2026

Rebuilding *BearcastMedia.com*

Architecture, content modeling, integrations, and deployment of a student-media platform — from inherited WordPress to a headless, CMS-driven, edge-deployed system.

AUTHORED BY

Cam Garrison
for Bearcast Media

GITHUB

@notChewy1324
camgarrison.com

PUBLICATION

May 2026

Contents

§ 01 — ABSTRACT

Abstract & Executive Summary

> *This document describes the complete technical redesign of **BearcastMedia.com**, the public-facing website of a 25-year-old independent student-media organization. The new build replaces an inherited WordPress installation with a static, multi-page front end deployed to Cloudflare Pages, backed by a custom-modeled Sanity Studio for content and integrated with Radio.co for live audio streaming.*

The objective of the rebuild was not novelty. It was control — over performance, over editorial workflow, over the long-term operating cost of running a student organization's web presence, and over the developer experience for the volunteers who will inherit this codebase after the author graduates. The inherited site was not a custom build. It was a generic WordPress template, adopted years earlier by a previous student cohort who needed a public presence quickly and did not have the technical background to author one. The organization had long expressed dissatisfaction with the template and a desire to migrate to something authored specifically for Bearcast — this project is that migration.

The replacement architecture decouples the concerns the template conflated. Content lives in **Sanity** as a structured, schema-validated dataset addressable over an HTTPS API. The public site is a small set of hand-authored HTML pages with a single shared stylesheet and a single shared JavaScript file. Those pages are served as static assets from **Cloudflare's edge network**, fetching live data from Sanity at runtime in the browser. Audio streams from **Radio.co** via their public widget and JSON metadata endpoint, with a custom “Now On Air” component computing the current show from the CMS-defined schedule.

The result is a site that loads on first paint without waiting on a database round-trip, edits content through a purpose-built studio rather than a generic admin panel, costs effectively zero dollars per month to operate, and presents a codebase small enough that a successor can read every meaningful line in an afternoon.

Key Decisions at a Glance

CONCERN	DECISION	REASON
Rendering model	Static HTML + client-side fetch	No build step, no framework lock-in, trivial to debug.
Content layer	Sanity (hosted, schema-defined)	Structured content, custom desk navigation, free tier covers our scale.
Hosting	Cloudflare Pages	Free, global edge, HTTPS & DDoS protection by default.
Audio	Radio.co widget + JSON API	Existing station infrastructure; no self-hosting of audio.
Departments list	Hardcoded in `scripts.js`	Stable taxonomy; CMS edits here would create routing bugs.

§ 02 — CONTEXT

Project Context & Inherited State

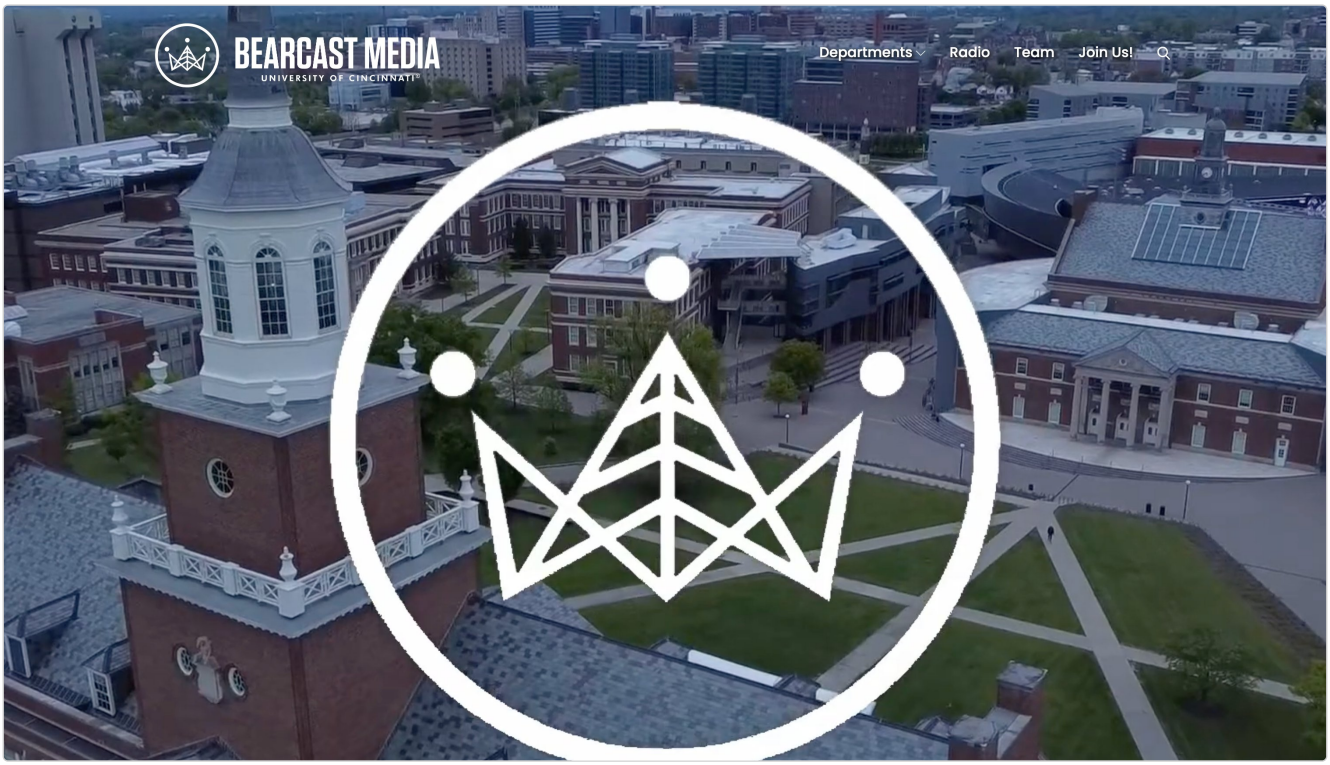
Bearcast Media is a student-run media organization founded in 1999, operating five departments: Radio, Sports, BTV (television), Journalism, and Music. The organization has continuously published, broadcast, and aired content for more than a quarter century. The website is one of several public surfaces — alongside the live radio stream, the broadcast schedule, and various social accounts — but it is the only surface that is fully under the organization's editorial control and the only one searchable from outside.

2.1 The Inherited Site

The site inherited by the author at the start of this project was a generic WordPress installation running a third-party template. It is important to note up front what the inherited site was and was not. It was not a custom build authored for Bearcast Media. It was a commercial multi-purpose theme — the kind of catch-all template sold for several common use cases at once (small business, media, blog, education) — pressed into service by a previous cohort of students who needed a public web presence quickly and lacked the technical background to design and author one from scratch. The decision was pragmatic at the time: a template-and-plugins approach is the standard solution for organizations in exactly this position, and WordPress backs a significant portion of the live web for the same reason.

The cost of that pragmatism, however, accrued over time. The organization's directors had expressed for several years a desire to migrate off the template. The reasons were both aesthetic — the template's voice was generic where Bearcast's is deliberately editorial — and structural. The structural reasons matter most for this paper and are enumerated below:

- **Plugin sprawl.** Functionality was distributed across a long tail of third-party plugins of varying maintenance status. Each plugin carries its own update cadence, its own security obligations, and its own database footprint. WordPress documents this as a known operational concern — its own ecosystem statistics show plugin vulnerabilities as one of the leading vectors for compromise in the platform.
- **Theme entanglement.** Editorial layout, branding, and underlying display logic were co-located inside the theme's PHP templates. This is the standard WordPress model — themes are responsible for rendering — but it means a content change risks a layout regression and vice versa. Decoupling content from presentation is what motivates the broader headless-CMS movement, and is the central architectural separation the rebuild adopts.
- **Operational overhead.** WordPress core, plugin, and PHP version updates required active stewardship from a student maintainer — a role with annual turnover. A web presence that needs regular patching cannot be maintained by a rotating volunteer staff without continuity loss.
- **Editorial mismatch.** The WordPress admin exposed a generic post/page model with category and tag taxonomies. A student newsroom does not think in “posts.” It thinks in *desks*, *shows*, *games*, *episodes*, and *reviews* — categories that WordPress can be coerced into representing via custom post types or ACF (Advanced Custom Fields), but does not natively model out of the box. The cumulative editorial friction of forcing one model onto the other compounds over years.
- **Stated organizational preference.** The most important reason: the directors and editorial team had stated, repeatedly, that they did not want to be on WordPress and wanted a site authored for them. The rebuild was therefore not technology-pull but organization-pull — the brief existed before the technical approach was selected.



INHERITED WORDPRESS SITE — HOMEPAGE HERO

Figure 2.1 — The inherited bearcastmedia.com homepage as rendered by the third-party WordPress template. The University of Cincinnati lockup is integrated into the masthead by the template, not by editorial choice; the dropdown-style departments menu collapses five distinct editorial surfaces behind a single hover target.

2.2 Constraints on the Rebuild

Any replacement had to satisfy several constraints specific to a student organization, not a commercial product team:

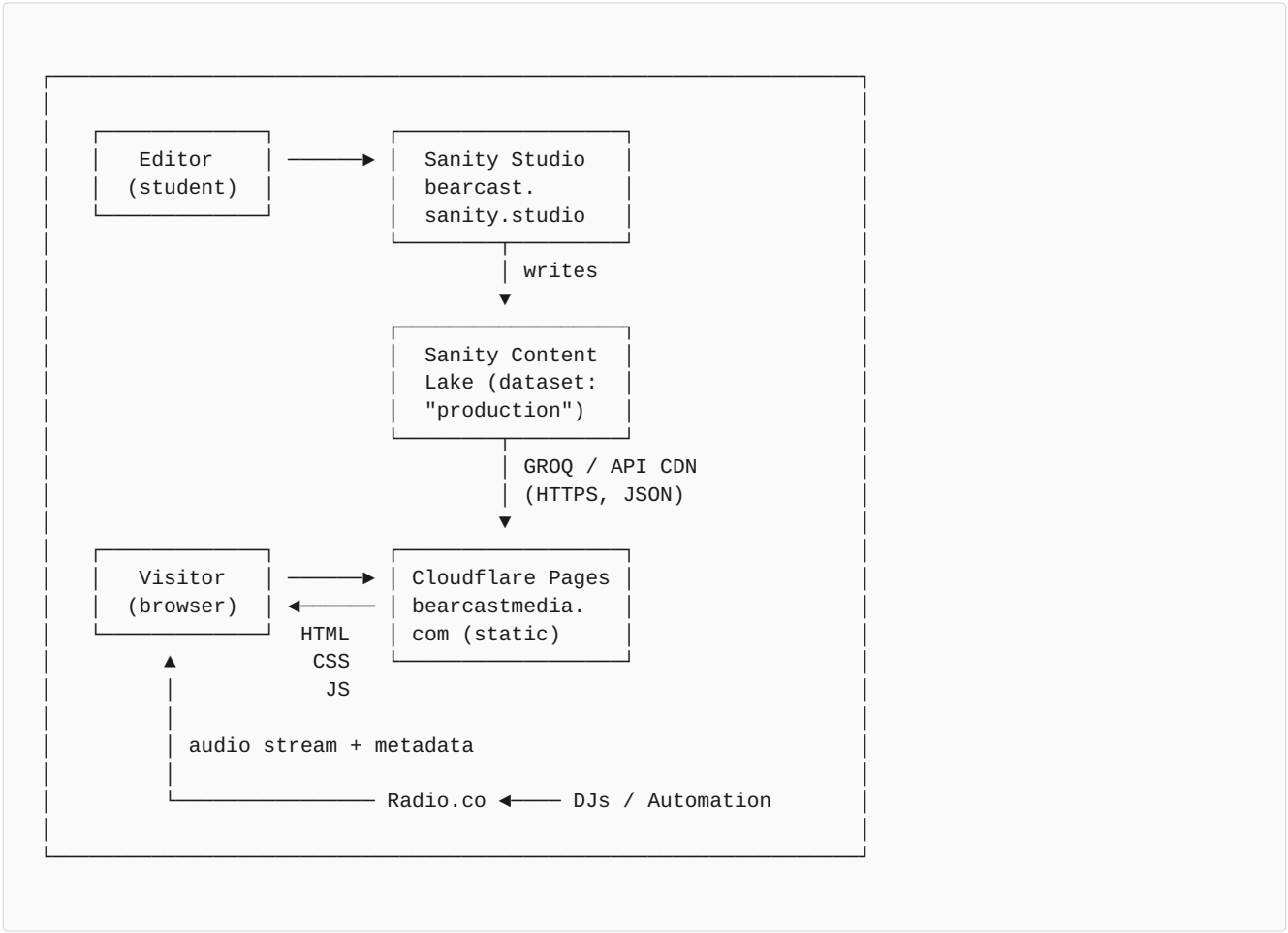
1. **Zero or near-zero ongoing cost.** Budgets are reallocated annually and cannot be assumed to persist.
2. **Hand-off survivability.** The author will graduate. The next maintainer is unlikely to be a senior engineer. The codebase must be readable and operable by a motivated student with web fundamentals.
3. **Editorial accessibility.** Editors, who rotate yearly and across departments, must be able to publish without engineering intervention.
4. **Brand fidelity.** The Bearcast voice — independent, professional, deliberately editorial — must come through in the design rather than be papered over by a generic theme.
5. **No vendor traps.** Where possible, content must be portable. The Sanity dataset is the durable artifact; the frontend is replaceable.

> **DESIGN PRINCIPLE**
>
> Treat the content layer as the durable asset and the rendering layer as disposable. A future maintainer should be able to discard the entire `scripts.js` file and rebuild a new frontend against the same Sanity dataset without losing a single article, show, or director profile.

§ 03 — ARCHITECTURE

System Architecture Overview

The system is composed of four cleanly separable layers, each owned by a different vendor or hosted independently. None of the layers know the implementation details of any other; they communicate exclusively over public HTTPS endpoints.



3.1 Research & Stack Selection

The architecture described in the rest of this section was not the first option considered; it was the option that survived a deliberate research and evaluation phase undertaken before any production code was written. The author surveyed the current state of practice for editorially-driven, low-budget, long-maintained web properties — drawing on documented public case studies from comparable student-media and small-publisher organizations, vendor documentation for the relevant CMS and hosting candidates, and the broader headless-CMS and Jamstack literature published over the last several years. Three architectural families emerged as plausible:

1. **A) Stay on WordPress, switch templates and modernize.** Adopt a current, well-maintained theme framework (e.g. Sage / Roots) and consolidate the plugin footprint. This option was the lowest-effort technically but directly contradicted the organization's stated goal of moving off WordPress.
2. **B) Headless WordPress with a JavaScript frontend.** Continue using WordPress as the content store, but consume its REST or GraphQL API from a Next.js or similar frontend. This retains WordPress's editor experience while

modernizing the rendering layer. It also retains every operational obligation of the WordPress side and adds a new one (the frontend). For a student team this is two systems to maintain where there was one.

3. **C) Headless CMS (Sanity / Contentful / Strapi / Payload) with a static or near-static frontend.** Migrate content into a purpose-built structured-content platform; serve a static or hybrid frontend from a CDN. This is the dominant pattern in current independent-publisher architecture (the Jamstack family) and is what the rebuild ultimately adopted, with Sanity selected as the CMS layer.

The selection among CMS candidates was driven by four requirements: (1) a free tier that comfortably accommodates a student-media organization's content volume and editor count, (2) first-class support for schema-defined structured content (not just “posts and pages”), (3) a configurable editor interface so the studio could be reorganized around the newsroom rather than around document types, and (4) a content API portable enough that the Bearcast dataset would survive being decoupled from any one frontend.

Sanity satisfied all four constraints, and uniquely so on the third: its desk-structure API (documented under `sanity/structure`) is the most flexible studio-layout system in the headless-CMS field, and is what enables the department-grouped navigation described in §4.5. The competing platforms either lock the admin UI layout (Contentful), require self-hosting (Strapi, Payload) — which reintroduces the operational obligation the project was trying to eliminate — or charge for the seat counts a student organization actually needs.

For hosting, the static-asset frontier was similarly surveyed across Cloudflare Pages, Netlify, Vercel, and GitHub Pages. The choice of Cloudflare Pages is justified in §09; the short summary is that for a static site at this scale, all four options would have worked, and Cloudflare was selected for the combination of free tier, global edge, and integrated networking primitives the organization may want later (Workers, R2, Turnstile).

3.2 The Four Layers

1. **Content layer — Sanity.** A hosted, schema-defined content store. Every editable piece of the site (articles, reviews, game coverage, BTV episodes, radio shows, people, departments, GBM meetings, site-wide settings) lives here as a typed document. The schema is version-controlled in the codebase; the data is hosted by Sanity.
2. **Editorial surface — Sanity Studio.** A custom-configured React app published to `bearcast.sanity.studio`. It is the only place where content is created or edited. The left-hand navigation (“desk structure”) is modeled on a student newsroom and intentionally diverges from Sanity's default alphabetical list of document types.
3. **Presentation layer — Static HTML on Cloudflare Pages.** The pages themselves are seven hand-authored HTML files (`index` , `radio` , `sports` , `btv` , `journalism` , `music` , `team` , `join` , `404`), a single `styles.css` , and a single `scripts.js` . There is no build step. There is no framework. The files shipped to the edge are the files in the repo.
4. **Audio layer — Radio.co.** Live audio and current-track metadata come from Radio.co's player widget and station-info endpoint. The schedule grid and “Now On Air” component are independently driven by the Sanity `radioShow` dataset; Radio.co is the transport for the actual sound and now-playing track.

3.3 Why Static-First

A common alternative would have been a server-side-rendered framework — Next.js, Astro, or similar — fetching Sanity content at build time and producing pre-rendered HTML. That approach is well-understood and would have produced a fast site. It was deliberately not chosen, for three reasons:

- **Build coupling.** Every content publish would either require a rebuild (operationally fragile for a student org) or a webhook + revalidation system (more moving parts to maintain).

- **Hand-off friction.** A successor maintainer landing in a Next.js project must learn the framework before they can change a button color. A successor landing in `journalism.html` can read it top to bottom.
- **Real bottleneck.** The actual rendering of a Bearcast page is not the bottleneck. First paint is dominated by font loading and a single Sanity fetch, both of which a framework would not eliminate.

The cost of the static-first choice is that listings load in two phases — a skeleton render, followed by content arrival. The callout in §08 discusses how that's mitigated.

§ 04 — CONTENT LAYER

Sanity CMS — Content Model & Studio

> *Sanity is the durable artifact of this project. If everything else were thrown away tomorrow, the contents of the Sanity dataset would still represent a complete, structured record of every article, show, episode, and person Bearcast has published.*

4.1 Project & Dataset

The Studio is configured against a single Sanity project (ID `3getuuyu`) and a single dataset (`production`). These values are public — they identify *which* content store the client is asking about, not *whether* the client is authorized to write to it. Write access is gated by Sanity's own authentication; the frontend never holds a write token.

SANITY.CONFIG.TS — EXCERPT

```
export default defineConfig({
  name: 'bearcast-studio',
  title: 'Bearcast Media',
  projectId: '3getuuyu',
  dataset: 'production',
  plugins: [
    structureTool({structure: deskStructure}),
    visionTool({ defaultApiVersion: '2024-01-01' }),
  ],
  schema: { types: schemaTypes },
})
```

4.2 Content Modeling Approach

Before any document types were written, the dataset was modeled on paper following the content-modeling discipline established by practitioners such as Sara Wachter-Boettcher and Mike Atherton (see *Designing Connected Content*, and the recurring treatment of structured content in *A List Apart* and similar publications). The central principle: a content model should describe the things the organization actually publishes, not the pages those things eventually live on. “Page” is a presentation concept; “Article,” “Game Recap,” “Episode,” and “Person” are content concepts.

Concretely, this means the Bearcast schema rejects the WordPress instinct to express every difference as a category or tag on a single “Post” type. A game recap has structurally different metadata (sport, matchup, final score, venue) from an opinion column; a music review has structurally different metadata (album, artist, score, verdict, cover art) from a game recap. The schema therefore promotes these differences to distinct document types — `article`, `review`, `coverage`, `episode`, `radioShow` — each with its own validation rules and its own editorial card preview. Cross-cutting concerns (an article that *also* belongs on the music page) are expressed via fields on the document, not by stuffing them into a folksonomy of tags.

Two reusable nested types (`timeSlot` and `richBody`) are extracted because they recur across documents. `timeSlot` appears once per show airing inside a `radioShow`; `richBody` is the Portable Text body used by every long-form document. Extracting them prevents drift — a change to the body editor propagates everywhere automatically.

4.3 Document Types

The schema is intentionally narrow. Nine document types and two reusable object types cover every editable surface on the public site:

TYPE	KIND	PURPOSE
`siteSettings`	Singleton document	Mission, tagline, hero copy, social links, recruiting toggle, footer.
`article`	Document	Long-form journalism. `desk` field routes to Journalism / Sports / Music pages.
`review`	Document	Music reviews — score, verdict, album, artist, cover art.
`coverage`	Document	Game recaps — sport, matchup, final score, venue, game date.
`episode`	Document	BTV video episode bound to a YouTube ID with regex validation.
`radioShow`	Document	Recurring radio program with one or more weekly `timeSlot` entries.
`person`	Document	A single human. Referenced as author, host, or director.
`dept`	Document (legacy)	Department metadata. Frontend currently uses hardcoded list; schema retained.
`gbm`	Document	General Body Meeting — date, time, location, topic, slides URL.
`timeSlot`	Object	Nested in `radioShow`. Day + 24h start/end with regex validation.
`richBody`	Object	Portable Text body — used by article, review, coverage.

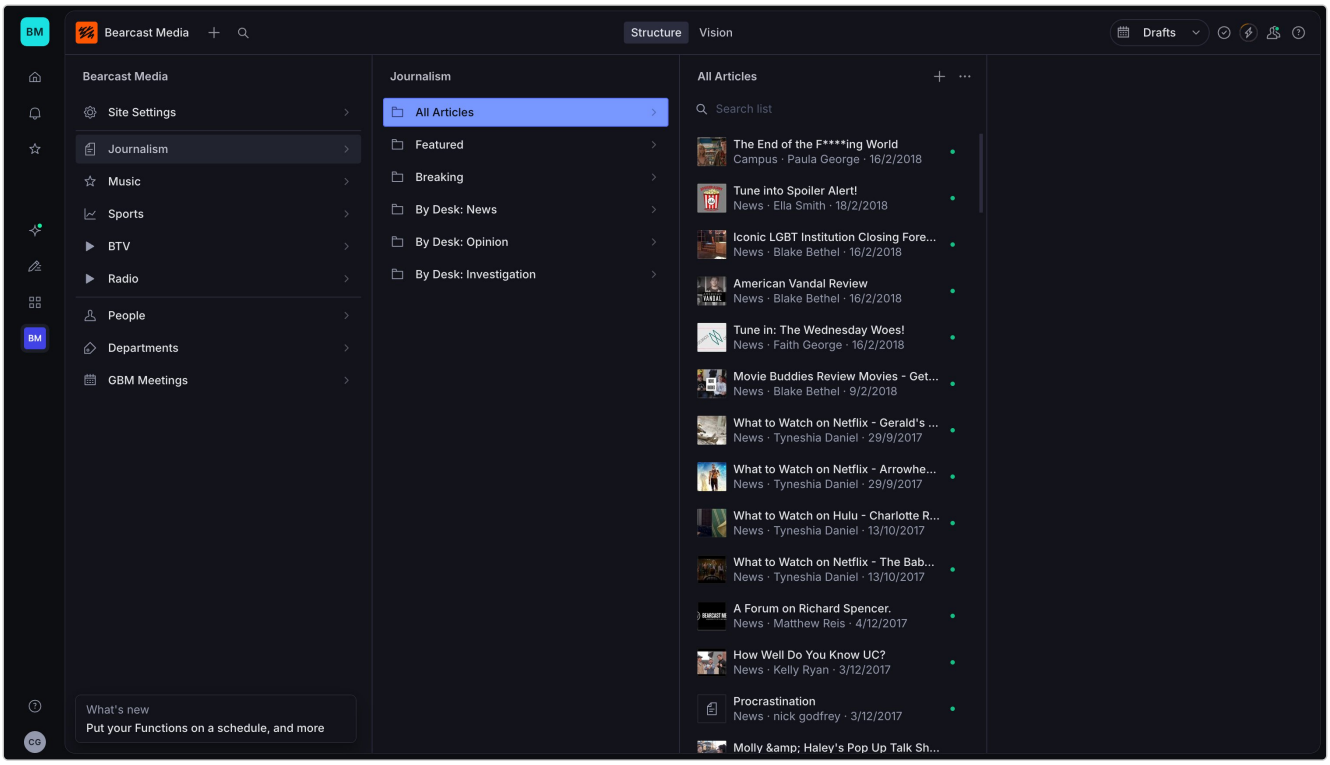
4.4 References — Why One `person` Document

A naive content model would store author names as strings on each article. The current model promotes `person` to its own document type and uses Sanity's reference field type to point other documents at it. The benefit is concrete: there is exactly one Maya Chen document. Update her bio, photo, or pronouns once, and every article she's written, every show she hosts, and her director card all update simultaneously.

The `person` schema also encodes the difference between a director and a contributor (`isDirector: boolean`), and between general directors and executive leadership (`isExecutive: boolean`) — both used by the desk navigation to populate filtered views.

4.5 The Custom Desk Structure

Sanity's default left-hand navigation lists all document types alphabetically. For a newsroom that is hostile to editorial flow — “Coverage” lands before “Episode” lands before “GBM”, none of which is how a sports reporter or a BTV producer thinks. The custom `deskStructure` reorganizes the studio around departments instead of types:



SANITY STUDIO — CUSTOM DESK NAVIGATION

Figure 4.1 — The custom desk navigation. The left column lists departments instead of document types; clicking Journalism opens the middle column with All Articles, Featured, Breaking, and per-desk filtered views; clicking one opens the article list on the right.

Inside each department, sub-items use GROQ filter strings to slice the same underlying type into editorially meaningful views. For example, `article` appears under Journalism three times — "All," "Featured" (`featured == true`), and "Breaking" (`breaking == true`) — and again under Music and Sports filtered by `desk == "MusicJournalism"` and `desk == "SportsJournalism"` respectively.

4.6 Singleton Enforcement

The `siteSettings` document is a singleton — there must be exactly one. Sanity has no first-class concept of singletons, so the constraint is enforced at the studio configuration layer by filtering out the `duplicate` and `delete` document actions when the schema type is `siteSettings`:

SANITY.CONFIG.TS — SINGLETON ENFORCEMENT

```
document: {
  actions: (prev, context) => {
    if (context.schemaType === 'siteSettings') {
      return prev.filter(
        ({action}) => !['duplicate', 'delete'].includes(action),
      )
    }
    return prev
  },
}
```

4.7 Portable Text — The `richBody` Object

Long-form bodies (articles, reviews, recaps) are stored as Portable Text — Sanity's structured JSON representation of rich content. The schema permits standard block styles (paragraph, H2–H4, blockquote), bullet and numbered lists, the usual inline decorators (bold, italic, underline, strikethrough, inline code), link annotations with optional new-tab behavior, inline images with required alt text, and a custom `pullQuote` object for editorial emphasis.

The frontend serializer lives in `scripts.js` (`renderPortableText`) and is intentionally small — Portable Text is portable precisely because the storage format is decoupled from any one renderer.

> **WHY PORTABLE TEXT VS. HTML IN A FIELD**
>
> Storing rich content as raw HTML is the most common antipattern in CMS work. It conflates presentation with content, makes re-theming a migration project, and turns every renderer upgrade into an HTML-sanitization audit. Portable Text stores the *structure* of an article — "this is a heading, this is a quote, this is a list" — and lets the renderer decide what those mean visually.

§ 05 — FRONTEND

Frontend Code & Page Architecture

The public site is built as a small set of static HTML pages sharing a single stylesheet and a single JavaScript file. There is no transpilation, bundling, or build step. The files on disk are the files served.

5.1 File Layout

FILE	ROLE
`index.html`	Homepage — hero, departments grid, “Now On Air” widget, recent content.
`radio.html`	Radio department — weekly schedule grid, show cards, live player.
`sports.html`	Game coverage feed with sport filter pills.
`btv.html`	BTV episodes — channel-strip filter, YouTube-embedded player.
`journalism.html`	The Bearcast — full newspaper masthead, desk filters, article modal.
`music.html`	Music — reviews with scores, music journalism articles.
`team.html`	Director grid populated from `person` documents.
`join.html`	Recruiting page — departments, roles, GBM schedule.
`404.html`	Custom 404 served by Cloudflare Pages.
`styles.css`	Single stylesheet for every page. Theme-aware via CSS variables.
`scripts.js`	Single JS file — fetches Sanity, renders content, handles UI state.

5.2 The Shared Navigation Component

Every page includes the same `<nav class="bc-nav">` block — a primary menubar for desktop and a slide-in panel for mobile. Active state is driven by a single `data-active="true"` attribute on the current page's link and a corresponding `aria-current="page"` for screen readers. The mobile panel is themed as a TV/radio “channel changer,” with each department numbered `CH·01` through `CH·05` — an intentional nod to the broadcast metaphor that runs through the brand.

5.3 The Theme System

Light/dark mode is implemented via a `data-theme` attribute on the root `<html>` element and CSS variables keyed off that attribute. The user's selection is persisted to `localStorage`.

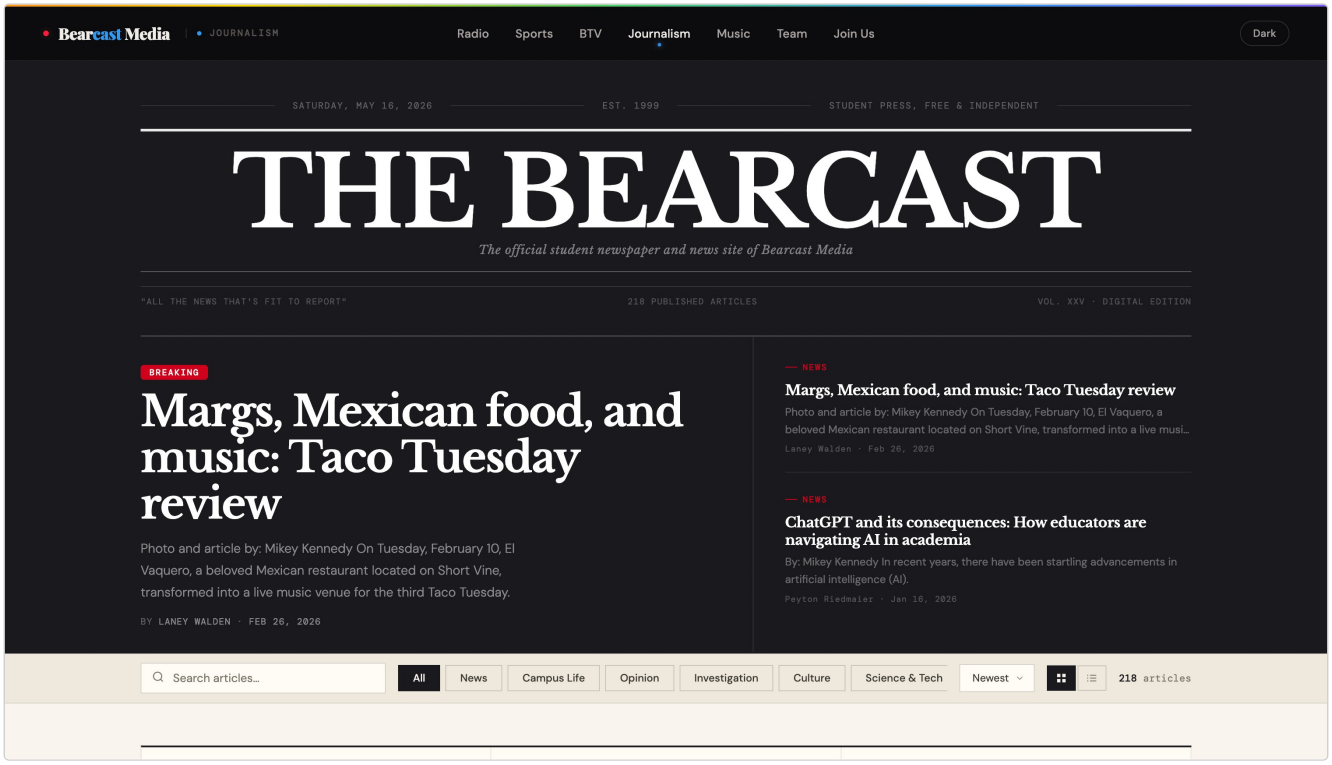
A subtle but important detail: the theme bootstrap script is inlined in the `<head>` and runs *synchronously, before first paint*. This is the only JavaScript on the page that *must* be inline and blocking. If it were deferred, the page would flash in the wrong theme for a frame and then snap to the correct one — a small but user-visible defect known as flash-of-incorrect-theme.

THEME BOOTSTRAP — RUNS PRE-PAINT

```
(function(){
  try {
    var saved = localStorage.getItem('bc-theme');
    var fallback = document.documentElement
                    .getAttribute('data-theme') || 'dark';
    var theme = (saved === 'light' || saved === 'dark')
                 ? saved : fallback;
    document.documentElement.setAttribute('data-theme', theme);
  } catch (e) { /* private mode etc */ }
})();
```

5.4 The Journalism Page — A Newspaper, Not a Blog

The Journalism page deliberately departs from the rest of the site's visual language and adopts a full newspaper treatment: masthead with edition line and volume number, hero with breaking-news label, desk filter pills, grid/list view toggle, sort selector, and a modal article reader with a reading-progress bar. This is not decoration — it's a design statement that **The Bearcast** is a newspaper, not a feed.



JOURNALISM PAGE — NEWSPAPER MASTHEAD, EDITION LINE, BREAKING-STORY HERO

Figure 5.1 — The Bearcast adopts a full newspaper treatment: nameplate, edition line, hero with breaking-news label, desk pills, sort selector, grid/list toggle, and an article-count badge.

5.5 Hardcoded Departments — A Deliberate Choice

The five Bearcast departments (Radio, Sports, BTV, Journalism, Music) are *not* driven from Sanity. They are constants in `scripts.js`. A `dept` schema exists in the studio but is unwired from the public site.

This is documented in `structure.ts` as a deliberate decision. The departments aren't editorial content; they're routing primitives — each one corresponds to an HTML file, a nav link, and a hardcoded accent color. Allowing editors to add or rename them from the CMS would silently break links the moment a page file did not exist to match.

> **TRADEOFF**
>
> The cost of this choice is that adding a sixth department requires a code change, not a CMS edit. Given that Bearcast has operated with the same five departments for years and reorganizes them on a multi-year timescale, the tradeoff is well worth it.

§ 06 — INTERACTION DESIGN

The Custom Desktop Cursor

> *The site replaces the operating system's default mouse cursor with a custom SVG pointer that reacts to context — scaling on interactive elements, contracting on click, fading on text input, and disappearing entirely on touch devices. It is the most visible single piece of frontend craft in the build.*

The cursor is not chrome. It is a deliberate piece of brand expression on a site about broadcast media — a tactile, almost analog feel laid over a static-first site. It also serves a practical purpose: in a multi-page site where listings, modals, schedule grids, and player controls are all interactive in different ways, a context-aware cursor makes affordances immediately legible without adding visual noise to the page itself.

6.1 Capability Gating — Who Sees It

Before anything else happens, the cursor mount logic asks the browser three questions via the CSS Media Queries Level 5 specification's interaction-media features. If any answer is wrong, the cursor never installs and the OS pointer remains in charge:

1. **Has the user requested reduced motion?** `matchMedia('(prefers-reduced-motion: reduce)')` — the standardized signal a user-agent emits when the operating system's reduce-motion accessibility preference is enabled. If true, the cursor is suppressed. This satisfies WCAG 2.1 Success Criterion 2.3.3 (Animation from Interactions, Level AAA), which requires that motion animation triggered by interaction can be disabled.
2. **Is this a fine pointer device?** `matchMedia('(pointer: fine)')` — phones, tablets, and most touch laptops report `(pointer: coarse)` and are correctly excluded. The distinction is from the CSS Media Queries Level 4 interaction-media spec and is the canonical way to detect input-device fidelity without user-agent string sniffing. Touch users never see a stray arrow chasing a finger they're not holding.
3. **Is one already mounted?** A defensive `getElementById('bc-cursor')` check prevents duplicate cursors during hot reloads or partial navigation.

> **ACCESSIBILITY — RESEARCH NOTE**
>
> Custom cursors are a recurring accessibility regression in surveys of web-craft sites (see e.g. the WebAIM annual accessibility reports and the body of work by Adrian Roselli on pointer-interaction patterns). The dominant failure mode in those surveys is not the cursor itself but the absence of capability gates: implementations that hide the OS cursor unconditionally, ignore `prefers-reduced-motion`, or fail to restore the text caret over input fields. Skipping `prefers-reduced-motion` in particular can trigger vestibular symptoms in affected users — well-documented in Val Head's work on web animation accessibility. The Bearcast cursor is fully opt-out at the OS level by design, not by afterthought.

6.2 The Pointer Itself

The cursor is a single inline SVG — twelve points describing a classic arrow pointer with a leftward-facing tail. Two paths are layered: a wider stroke path that creates a subtle outline against any background, and a fill path that carries the visible color. In dark mode the fill is near-white with a semi-transparent black stroke; in light mode they invert.

SCRIPTS.JS — CURSOR SVG AND MOUNT

```
var el = document.createElement('div');
el.id = 'bc-cursor';
el.innerHTML =
  '<svg viewBox="-1 -1 13 22">' +
    '<path class="arrow-stroke" d="M0 0 L0 16 L3.5 12.5 ' +
                               'L6.5 19 L8.5 18 L5.5 11.5 L10 11.5 Z"/>' +
    '<path class="arrow-fill"   d="M0 0 L0 16 L3.5 12.5 ' +
                               'L6.5 19 L8.5 18 L5.5 11.5 L10 11.5 Z"/>' +
  '</svg>';
```

Both paths share the same geometry — the stroke draws underneath the fill, so a 1px shadow appears around the entire shape against any backdrop without an actual drop shadow filter being applied. The result is a cursor that remains legible on photographs, dark headers, and white modals alike.

6.3 Movement & The Animation Frame

A cursor that lags behind the real mouse position feels broken in a way few users can articulate but all can sense. The cursor therefore tracks the pointer on every `mousemove` event, but updates its visual position only inside the next `requestAnimationFrame` callback — coalescing potentially dozens of mousemove events per second into one DOM write per frame.

MOVEMENT LOOP — RAF-BATCHED

```
var cx = -100, cy = -100, scheduled = false;
document.addEventListener('mousemove', function (e) {
  cx = e.clientX; cy = e.clientY;
  if (!scheduled) {
    scheduled = true;
    requestAnimationFrame(function () {
      scheduled = false;
      el.style.transform = 'translate3d(' + cx + 'px,' + cy + 'px,0)';
    });
  }
}, { passive: true });
```

Three details matter here, and each was selected after surveying the browser-performance guidance current at the time of build.

First, the `passive: true` flag tells the browser the handler will never call `preventDefault()`, which allows the input pipeline to short-circuit a permission check and avoid blocking the scroll thread. This pattern is documented in Chrome's Web Performance team writing on scroll-jank and is explicitly recommended by the W3C DOM events specification for high-frequency listeners. Second, the transform uses `translate3d` rather than `top`/`left` so the cursor is promoted to its own GPU compositor layer and the update never triggers layout or paint — only composite. This is the canonical “jank-free animation” pattern documented by Paul Lewis and others in the Chrome DevTools team's rendering-performance literature. Third, the cursor's CSS sets `will-change: transform` so the browser allocates the compositor layer in advance — without it, the first move would incur a layer-promotion cost and present a single dropped frame. The `will-change` property is used sparingly here (only one element on the page) because its overuse defeats its own purpose, a caveat the MDN reference makes explicit.

6.4 The State Machine

The cursor is in exactly one of four visual states at any moment. States are represented as CSS classes on the cursor element:

STATE	CLASS	VISUAL	TRIGGER
Default	(none)	Arrow at 1.0× scale	Over passive content
Hover	`bc-hover`	Arrow at 1.15× scale	Over `INTERACTIVE` match
Click	`bc-click`	Snaps to 0.78× with spring easing	Mouse button down
Text	`bc-text`	Fades to 0 opacity, scales to 0.6×	Over text input

State transitions are driven by `mouseover` and `mouseout` on the document. Two selector strings — `INTERACTIVE` and `TEXT` — list every selector the cursor cares about. When the event target matches or is contained within an interactive selector, the cursor enters `bc-hover`; when it matches a text selector, it enters `bc-text` and effectively disappears so the OS's native caret-aware text cursor takes over.

The interactive selector list is long and intentional. It covers standard semantic elements (`a`, `button`, `label`, `select`), every ARIA role that implies interaction (`role="button"`, `role="link"`, `role="tab"`, `role="menuitem"`, …), every named UI primitive in the Bearcast design system (`.art-card`, `.ep-card`, `.dir-card`, `.show-card`, …), every navigation pill and filter chip, every player control, and a generic catch-all of `[onclick]` and the project's data attributes (`[data-day]`, `[data-cat]`, `[data-dept]`, …). If a future contributor adds a new card class, the cursor will not automatically know about it — this is documented as a maintenance point in the code comments.

6.5 Hiding the OS Cursor — Conditionally

Once the custom cursor has successfully mounted, a single class (`bc-cursor-active`) is added to the root `<html>` element after a 280ms delay. The delay avoids flashing the OS cursor off in the brief window before the custom one is positioned. Two corresponding CSS rules then take effect:

CURSOR VISIBILITY RULES

```
html.bc-cursor-active *{
  cursor: none !important;
}
html.bc-cursor-active input,
html.bc-cursor-active textarea,
html.bc-cursor-active [contenteditable]{
  cursor: text !important;
}
```

The first rule hides the OS cursor everywhere. The second rule selectively reinstates the OS's `text` cursor on input fields — because a custom arrow over a text input is genuinely worse than no custom cursor at all. This is the cleanest possible escape hatch: in any context where the browser's native UI is unambiguously better (an input caret, a focus ring, an OS-level autocomplete), the native UI wins.

6.6 Edge Cases the Cursor Handles

- **Mouse leaves the window.** A `mouseleave` listener fades the cursor to opacity 0; `mouseenter` restores it. Users alt-tabbing or scrolling out of the window don't see a stranded arrow.
- **Click feedback.** `mousedown` adds `bc-click`, `mouseup` removes it. The CSS transition uses a cubic-bezier curve with overshoot (`0.34, 1.56, 0.64, 1`) — the cursor doesn't just shrink, it springs.
- **Theme changes.** The cursor's fill/stroke colors are scoped via `html[data-theme="light"]` selectors and update instantly when the theme toggle fires — no JavaScript intervention required.
- **Dynamic content.** Because `mouseover` is delegated to `document`, content injected later (article modals, lazy-loaded cards, dynamically rendered episodes) is fully covered without re-registering listeners.

6.7 Why This Is Worth The Code

A custom cursor is the kind of feature engineers cite as a warning sign — a place where craft frequently exceeds judgment. The case for it here is specific: Bearcast is a media brand where the public surface *is* the brand. The site sits next to broadcasts, podcasts, and television shows; it needs its own production value. The cursor is roughly 90 lines of JavaScript and a similar amount of CSS — small enough to read, debug, and remove if a future maintainer disagrees. It is documented inline. It degrades gracefully. And, importantly, it is the first thing every visitor sees move on the page.

§ 07 — AUDIO INTEGRATION

Radio.co Integration & Live-Schedule Logic

The radio stream is delivered by Radio.co, the existing station provider. The website does not host or transcode audio. Two Radio.co surfaces are consumed:

1. **The embeddable player widget.** An iframe that handles the actual audio decoding, buffering, and playback UI. This is the safest integration — the user's stream experience is identical to Radio.co's own player.
2. **The station-info JSON endpoint.** Returns now-playing track metadata, station status, and history. Polled on the homepage and radio page to populate the “currently playing” line under the Now On Air widget.

7.1 The Schedule Grid Is CMS-Driven, Not Radio.co-Driven

Radio.co has its own concept of a programmed schedule, but Bearcast's published schedule is authored in Sanity, on `radioShow` documents. There are two reasons for this: the Sanity-authored schedule is richer (host references, genre, accent color, tagline, featured flag), and the editorial team edits the schedule far more often than they touch the Radio.co console.

Each `radioShow` has a `timeSlots` array. Each `timeSlot` is a (day, startTime, endTime) tuple, with times stored as 24-hour strings (`"06:00"` , `"22:30"`) and validated by a regex that rejects anything outside `00:00–23:59` .

TIMESLOT.TS — SCHEMA VALIDATION EXCERPT

```
defineField({
  name: 'startTime',
  type: 'string',
  validation: (r) =>
    r.required().regex(/^([01]\d|2[0-3]):[0-5]\d$/, {
      name: 'time-format',
    }),
})
```

7.2 The Now-On-Air Computation

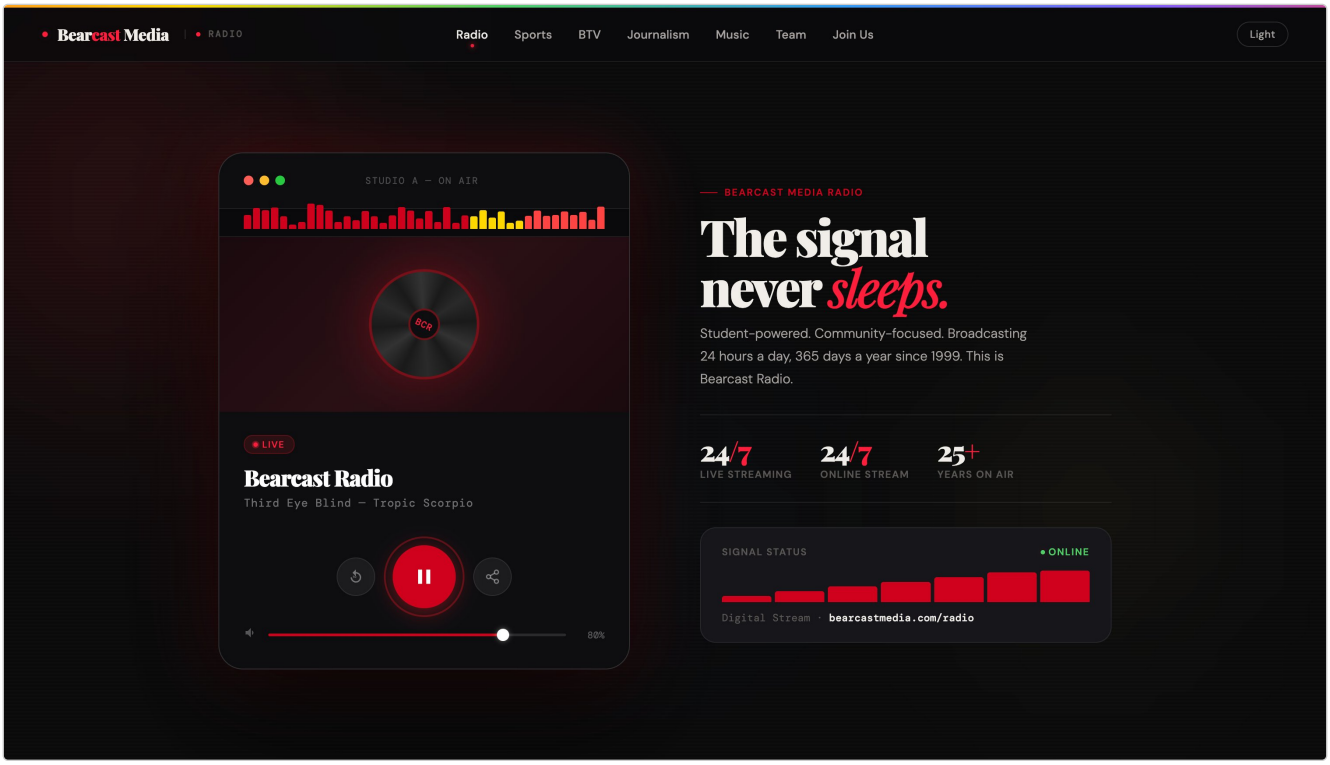
The homepage and radio page both surface a “Now On Air” widget. The computation is straightforward and runs entirely client-side:

1. Fetch all `radioShow` documents from Sanity.
2. Determine the current local day (`mon` – `sun`) and minutes-since-midnight.
3. For each show, scan its `timeSlots` for an entry where the day matches and the current minute falls within `[startTime, endTime)` .
4. Slots that cross midnight (e.g. `22:00 → 02:00`) are split into two checks against the calendar day boundary.
5. If multiple slots match — a content authoring error — the most recently published wins, with a console warning surfaced for editors.
6. If no slot matches, the widget falls back to the Radio.co now-playing track metadata.

Two technical decisions in that computation warrant elaboration, because they were arrived at after researching how comparable organizations (community radio stations, public-broadcasting schedule grids, podcast networks) handle the same problem and where their published implementations have gone wrong.

Local time, not UTC. Times are stored as wall-clock strings (`"06:00"`, `"22:30"`) without a timezone qualifier, and matched against the visitor's local day and minute. This is the correct choice for a station serving a single geographic community — a 6 AM show is 6 AM where the listeners and DJs are, regardless of the visitor's timezone. The alternative, storing slots as UTC moments, is the more common engineering instinct but produces a foreseeable failure mode: a viewer in another timezone sees a different "currently airing" show than the actual on-air state. Several reference implementations cited in discussion threads on the Sanity community and in published Jamstack tutorials make exactly this mistake.

Daylight saving handled by the browser, not the schema. Because times are wall-clock, the day-of-week and minute-of-day calculation uses the browser's `Date` object, which already incorporates the visitor's locale and DST state. The schema does not encode DST transitions and does not need to. The cost is that editors must not double-encode (e.g. by manually shifting a Sunday-morning show one hour every March); the schema's wall-clock validation regex helps enforce this by rejecting anything outside `00:00–23:59`.



BEARCAST RADIO — LIVE PLAYER, SIGNAL STATUS, STATION IDENTITY

Figure 7.1 — The radio page. The on-air widget renders the "Studio A — ON AIR" state; the play control sits centered on the BCR vinyl mark; signal status shows live broadcast level. The right column carries station copy and identity metrics.

7.3 Why Two Sources of Truth Works Here

A reasonable critique of this design is that the schedule lives in two places: Sanity (for display) and Radio.co (for automation). The justification is operational: when an editor changes the schedule in Sanity, the public-facing site reflects it immediately, without anyone needing access to the Radio.co console. When the actual audio routing changes in Radio.co, the public site keeps showing the intended editorial schedule until Sanity is updated to match. In a student organization where different people own different systems, this separation is a feature.

§ 08 — DATA FLOW

API & Data Flow

8.1 The Three External Endpoints

The browser talks to three external systems on a typical page load:

1. **Sanity Content Lake** — JSON content over the Sanity CDN. Read-only from the browser.
2. **Sanity CDN for images** — image URLs are constructed from Sanity asset references using the official URL-builder pattern, with width/quality/format parameters applied at request time.
3. **Radio.co** — station JSON for now-playing metadata, plus the iframe widget for audio.

8.2 Anatomy of a Page Load

Take the Journalism page as the worked example:

1. Cloudflare's edge serves `journalism.html`, `styles.css`, and `scripts.js` from cache. First paint occurs with the masthead, edition line, and a loading state for the article grid.
2. `scripts.js` issues a single GROQ query to Sanity for all `article` documents, projecting only the fields the page needs (headline, deck, excerpt, desk, author name, published date, cover image reference, emoji fallback, featured/breaking flags).
3. Sanity responds. The script renders the hero (the most recent `featured` or `breaking` article) and the grid (everything else, sorted by `publishedAt` descending).
4. Image references are converted to URLs at render time, with the requested dimensions baked in so Sanity's image CDN returns appropriately sized assets.
5. When the user clicks an article card, the corresponding Portable Text body is already in memory; the modal renders without a second network call.

8.3 Why Client-Side Fetching

The choice to fetch from the browser rather than at build time is a direct consequence of the static-first decision in §3. The cost is that the user sees a loading state for the duration of one HTTPS round-trip to Sanity's CDN. The benefit is that publishing requires no rebuild — the moment an editor hits “Publish” in the studio, the next visitor sees the change.

> **MITIGATION**
>
> The loading state is not blank. Every listing page renders a skeleton (masthead, filter bar, empty grid frame) immediately from HTML, then populates the grid when Sanity responds. The visible delay is the time-to-first-card, not time-to-first-paint.

8.4 GROQ vs. GraphQL

Sanity exposes two query languages: GROQ (Graph-Relational Object Queries — Sanity's native, designed specifically for document-database content shapes) and GraphQL (the industry-standard typed query language originated by Facebook and now governed by the GraphQL Foundation). The two are not equivalent, and the choice between them was not arbitrary.

GraphQL's strength is its type system and its rich tooling ecosystem; its weaknesses in this context are that it requires the schema to be re-deployed every time it changes (introducing a build dependency the static-first architecture is explicitly designed to avoid), and that its joins are awkward against the document-reference model Sanity uses internally. GROQ, by contrast, supports projections, joins, and filter expressions in a single readable string against the live dataset with no intermediate compilation step. Crucially, the desk-structure filters in `structure.ts` (§4.5) are already authored in GROQ — meaning the same query syntax describes both what the editor sees in the studio and what the frontend fetches at runtime. Keeping one query language across the studio and the frontend removes a category of cognitive overhead that would otherwise require future maintainers to learn two languages to debug a single piece of content behavior.

§ 09 — DEPLOYMENT

Cloudflare Pages Deployment

The site is hosted on Cloudflare Pages. Pages serves static assets from Cloudflare's global edge network with TLS, HTTP/3, Brotli compression, and DDoS protection included by default. There is no origin server.

9.1 Why Cloudflare Pages

PROPERTY	VALUE
Cost	$0 / month for our traffic volume
TLS	Automatic certificate issuance & renewal
CDN	Global edge, no separate configuration
Build pipeline	Git push → live in < 1 minute
Preview deploys	Every branch & pull request gets a unique URL
Rollback	One-click revert to any prior deploy
Custom 404	`404.html` served automatically

9.2 The Deploy Pipeline

Because the codebase has no build step, the Cloudflare Pages project is configured with no build command and a publish directory pointed at the repo root. A push to `main` triggers a deploy; the deploy is a CDN cache primer, not a compilation. End-to-end time from `git push` to live-on-edge is consistently under a minute.

9.3 The Studio Deploys Separately

The Sanity Studio is its own deployment target. Running `sanity deploy` publishes the studio to `bear-cast.sanity.studio` — the free `*.sanity.studio` subdomain that comes with every Sanity project, configured in `sanity.cli.ts` and kept entirely separate from the public site so editor-only JavaScript never ships to ordinary visitors.

§ 10 — COMPARISON

Before & After — WordPress vs. The New Build

> *This section sets the inherited WordPress site beside the current build along every axis a maintainer or editor cares about. The point is not to disparage WordPress — it remains an excellent fit for many sites — but to make explicit which constraints drove the rebuild and what changed.*

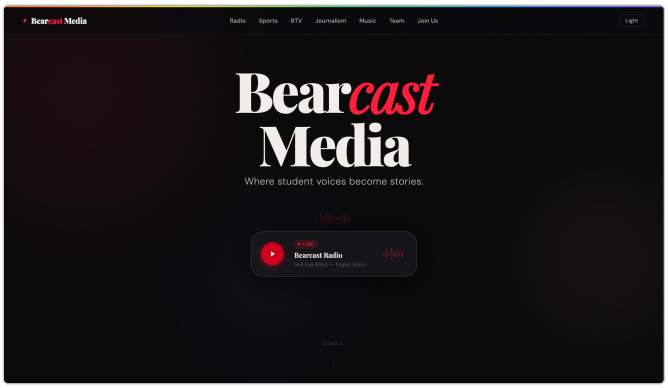
10.1 Architecture Side-by-Side

CONCERN	BEFORE — WORDPRESS	AFTER — SANITY + STATIC + CF
Build provenance	Third-party multi-purpose WordPress template	Authored end-to-end for Bearcast Media
Rendering model	PHP template rendering on every request	Static assets on Cloudflare edge; no origin
Content model	Generic posts & pages, coerced via categories/ taxonomies/plugins	Nine purpose-built document types with field-level validation
Editorial UI	Generic WP admin dashboard	Custom desk navigation, grouped by department
Theming	PHP theme files entangled with content logic	Single `styles.css` + single `scripts.js`
Patching obligation	WP core + every plugin + PHP version	None at runtime; Sanity is hosted
Build step	None for content; theme changes require FTP/ dashboard edit	None at all — files served as authored
TLS	Manual / plugin-driven	Automatic via Cloudflare
DDoS / WAF	Plugin or paid add-on	Included by default
Deploy / rollback	FTP edit, manual backup, manual revert	Git push; one-click rollback to any prior deploy
Hand-off readability	PHP + theme conventions + plugin landscape	HTML / CSS / JS + one TypeScript schema folder
Operating cost	Recurring WordPress hosting + premium plugin / template licenses	$0 within free tiers at current scale

10.2 Visual Comparison — Homepage

The homepage is the first surface every visitor sees and the surface that has to carry the most editorial identity. The inherited version (left) leads with a dropdown departments menu over a hero video; the new build (right) presents departments as direct, named entry points with the Now-On-Air radio widget promoted to first-paint priority.

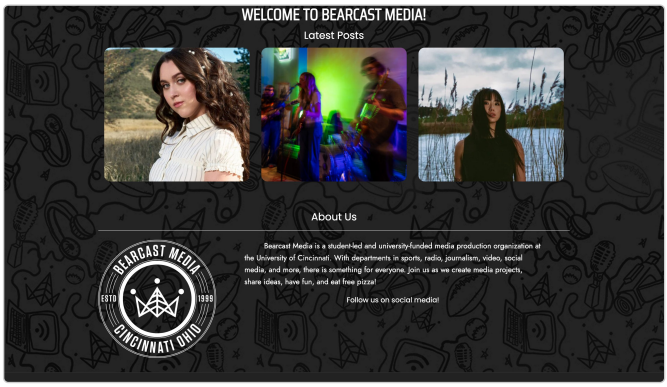
BEFORE — WORDPRESS



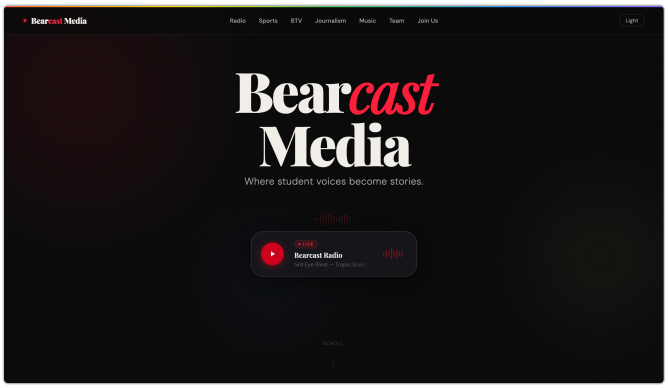
AFTER — CURRENT BUILD

Figure 10.1 — Homepage, before and after.

The second homepage panel — “Latest Posts” — illustrates the editorial mismatch described in §2. The inherited layout treats every department's output as an undifferentiated chronological feed of three thumbnails. The new build splits these into department-specific listings on their respective pages, each with desk-appropriate filtering and metadata (review scores, game results, breaking-news flags) rather than a single grid of unattributed images.



BEFORE — “LATEST POSTS” ON THE INHERITED SITE

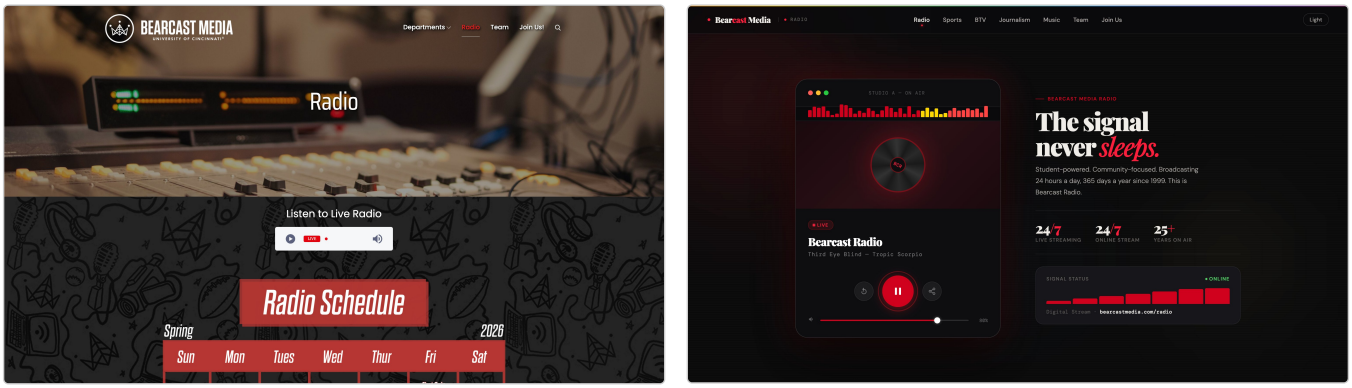


AFTER — NEW HOMEPAGE OPENS TO DIRECT DEPARTMENT NAMES

Figure 10.2 — Front-page editorial structure, before and after.

10.3 Visual Comparison — Radio

Radio is Bearcast's flagship product — the live station has aired continuously since the organization's founding. The inherited radio page treats the schedule as a static graphic (a red, hand-stylized “Radio Schedule” banner over a generic seven-day grid). The new build computes the schedule from the Sanity `radioShow` dataset as described in §07, includes a true Now-On-Air widget driven by the same data, and exposes the Radio.co stream through a single styled player.



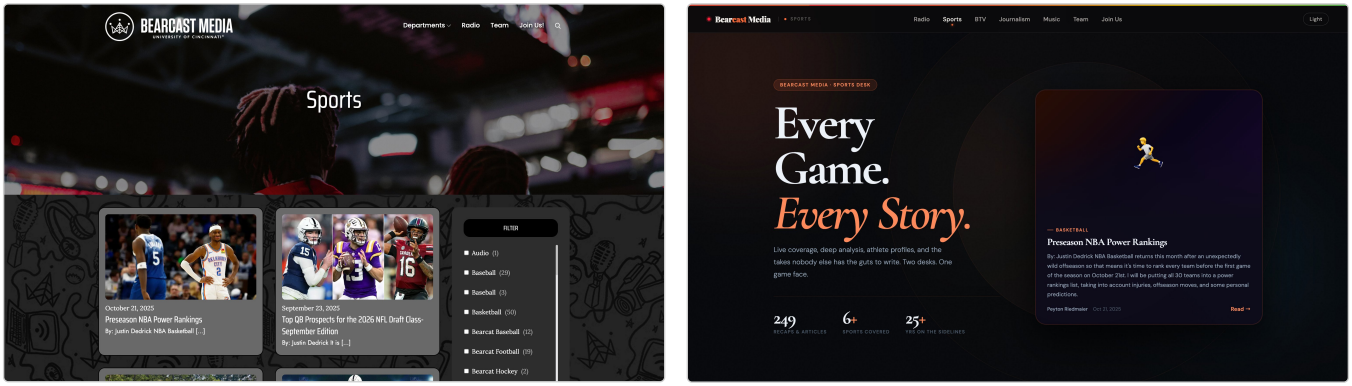
BEFORE — WORDPRESS RADIO PAGE

AFTER — NEW RADIO PAGE

Figure 10.3 — Radio surface, before and after.

10.4 Visual Comparison — Sports

Sports coverage on the inherited site (below, left) was a card grid with a long, sport-only sidebar filter — useful, but incapable of distinguishing a game recap (with score, opponent, venue) from a written feature. The new `coverage` document type captures that structured metadata directly (§4), enabling the new sports page to show finals, matchups, and venues as primary affordances on every card.



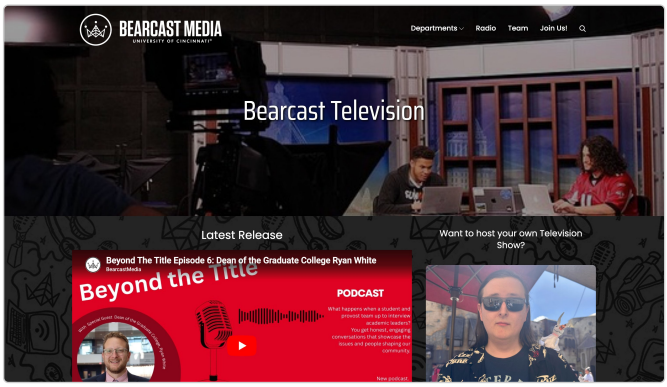
BEFORE — WORDPRESS SPORTS LISTING

AFTER — NEW SPORTS PAGE

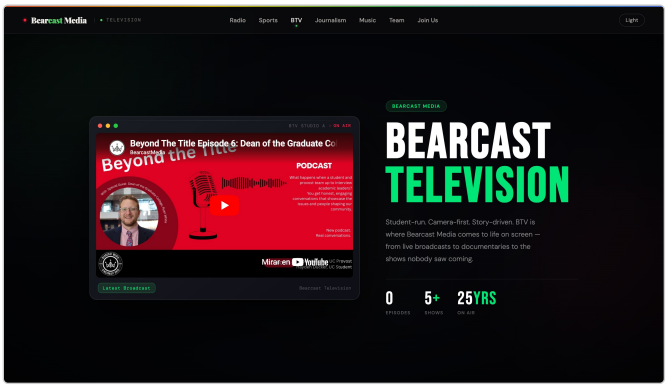
Figure 10.4 — Sports surface, before and after.

10.5 Visual Comparison — BTV (Television)

BTV is the television department. The inherited page exposed a single “Latest Release” embed alongside an unrelated “Want to host your own Television Show?” recruiting prompt. The new build models each release as an `episode` document with show name, season, episode number, runtime, and a regex-validated YouTube ID (§4) — enabling a proper episodes index, a channel-strip filter by show type, and structured metadata per release.



BEFORE — WORDPRESS BEARCAST TELEVISION PAGE

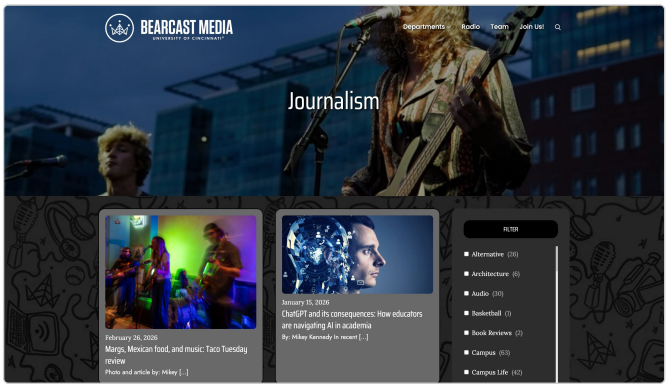


AFTER — NEW BTV PAGE

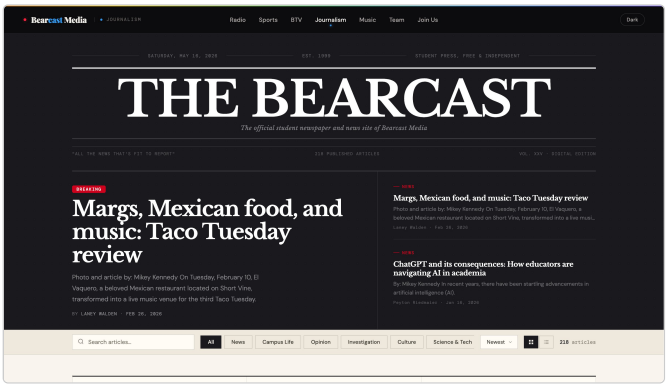
Figure 10.5 — BTV surface, before and after.

10.6 Visual Comparison — Journalism

The journalism page received the most aggressive redesign. The inherited page was a generic two-card grid with a sidebar filter of dozens of WordPress taxonomies (Alternative, Architecture, Audio, Basketball, …) — useful in principle but flatly unstructured for a newsroom. The new build, described in §5.4, adopts a full newspaper treatment: nameplate, edition line, breaking-news hero, desk pills, grid/list toggle, and a modal article reader.



BEFORE — WORDPRESS JOURNALISM LISTING

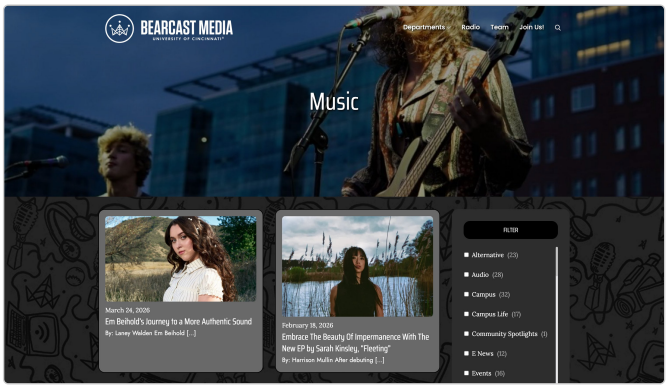


AFTER — “THE BEARCAST”

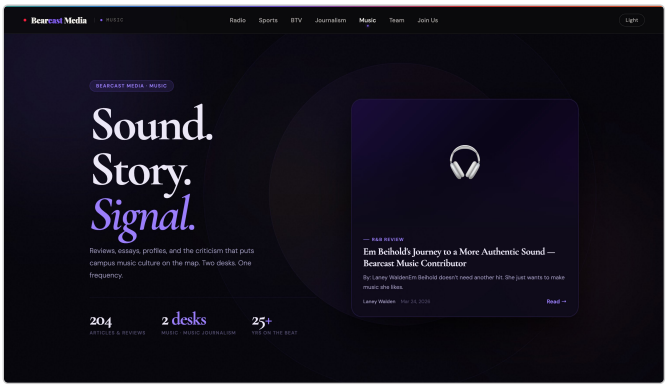
Figure 10.6 — Journalism surface, before and after.

10.7 Visual Comparison — Music

The inherited music page mixed reviews, features, and journalism into a single grid — Em Beihold's review sat visually identical to a Sarah Kinsley feature on her EP. The new schema separates `review` documents (score, verdict, album, cover art) from `article` documents flagged `desk == "MusicJournalism"`, so the new music page can render scored reviews distinctly from written journalism while sharing a single editorial surface.



BEFORE — WORDPRESS MUSIC LISTING

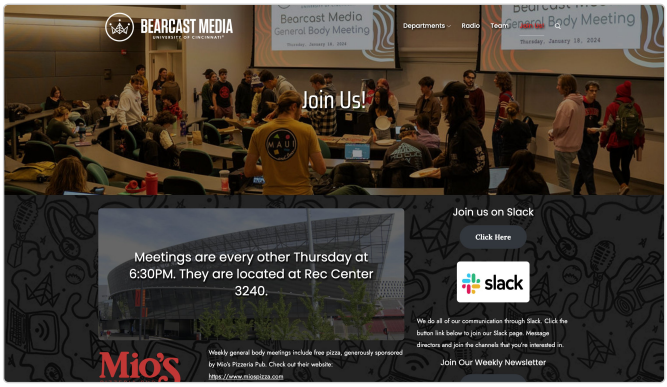


AFTER — NEW MUSIC PAGE

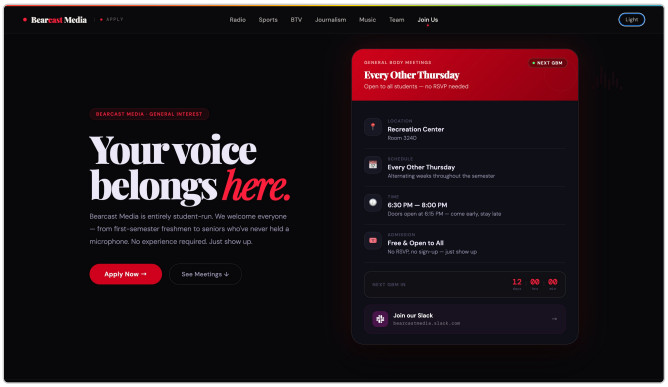
Figure 10.7 — Music surface, before and after.

10.8 Visual Comparison — Recruiting (Join Us)

The inherited Join Us page mixed a static meeting time ("every other Thursday at 6:30 PM" hardcoded into an image overlay) with a Slack invite. The new build sources meeting dates from the `gbm` document type, the department roster from `dept`, and the recruiting toggle from `siteSettings` — so the editorial team can move a meeting, mark one cancelled, or pause recruiting site-wide without engineering involvement.



BEFORE — WORDPRESS JOIN US PAGE



AFTER — NEW JOIN PAGE

Figure 10.8 — Recruiting surface, before and after.

10.9 Visual Comparison — Admin / Editorial Surface

Less visible to readers but most felt by the editorial team: the change in CMS. The inherited site used the standard WordPress `/wp-admin` dashboard. The new build uses a custom-configured Sanity Studio, with the desk navigation described in §4.5.

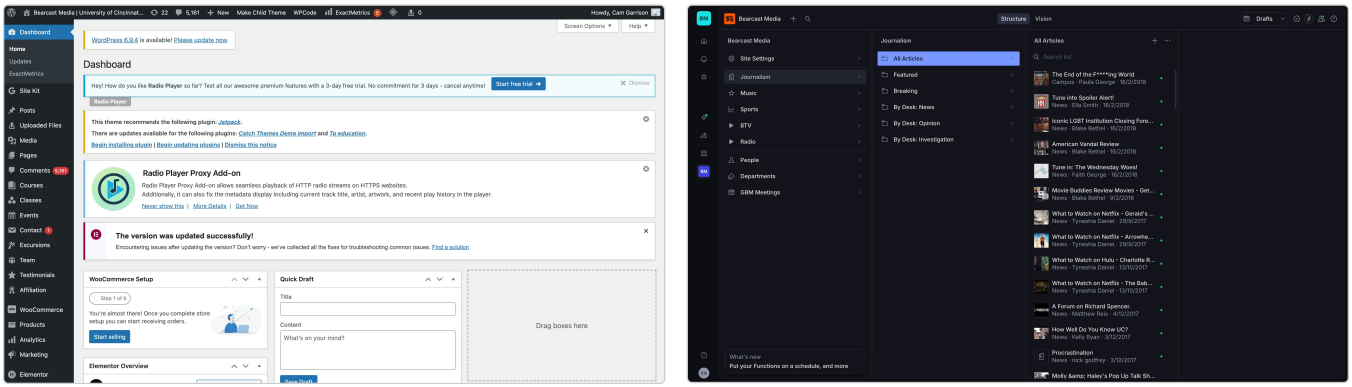


Figure 10.9 — Editor experience, before and after.

10.10 What Was Preserved

The redesign was deliberate about what to retain. The inherited template carried very little of what makes Bearcast Bearcast — the brand voice (independent, professional, deliberately editorial) lived in the organization itself, not in the template, and was treated as inviolable throughout the rebuild. The five-department structure, the 1999-founding-date provenance, the “Est. 1999” mark, and the “Vol. XXV” framing of the journalism page all carry forward institutional identity that predates the inherited template by many years. Replacing the technical layer was never intended to replace the organization's voice; if anything, it was intended to let that voice come through where the prior template had been flattening it.

Performance, Security & Operational Posture

11.1 Performance Profile

The site's performance characteristics follow directly from the architecture in §3:

- **Time to first paint** is bounded by edge latency to Cloudflare and parse time for a small HTML document — typically well under 200ms in North America.
- **Time to interactive** is bounded by the same plus the single Sanity content fetch. Sanity's CDN is similarly globally distributed.
- **Asset weight** is dominated by Google Fonts and Sanity-served images. The site's own CSS and JS are a small fraction of total page weight.
- **No render-blocking JS** on the critical path. `scripts.js` is deferred. The only inline script is the theme bootstrap, which is sub-1KB.

11.2 Security Posture

The site has no first-party server-side attack surface. Every request from a visitor terminates at Cloudflare's edge against static assets. There is no PHP runtime, no database connection, no admin login on the public domain. The studio lives on its own Sanity-managed subdomain and is gated by Sanity's authentication. This eliminates entire categories of risk that were intrinsic to the inherited stack.

Mapping against the OWASP Top 10 — the de facto standard reference for web-application risk — the relevant categories collapse as follows:

- **A01 Broken Access Control:** not applicable to the public site; there is no access-controlled surface to break. Studio access is delegated to Sanity's identity layer.
- **A02 Cryptographic Failures:** TLS terminated by Cloudflare with automatic certificate management, HTTP/3 and modern cipher suites enabled by default.
- **A03 Injection:** no SQL injection surface because there is no SQL. GROQ queries are constructed with parameters, not string concatenation. The Sanity SDK enforces this at the client layer.
- **A05 Security Misconfiguration:** dramatically reduced — there is no server to misconfigure. Cloudflare Pages exposes a small, declarative configuration surface (headers, redirects) versioned alongside the code.
- **A06 Vulnerable and Outdated Components:** the public site ships zero npm dependencies. The studio's dependencies are managed by Sanity's own release cadence.
- **A08 Software and Data Integrity:** deploys are immutable per-commit; rollback is one click to any prior deploy.
- **A09 Logging and Monitoring:** Cloudflare's edge analytics provide request-level observability without ingesting personally-identifying data on the public site.

The remaining OWASP categories (A04 Insecure Design, A07 Identification & Authentication, A10 SSRF) bear on the studio, not the public site, and are addressed by Sanity's platform-level controls rather than by application code in this code-base.

11.3 What Could Still Go Wrong

A complete operational posture is honest about its failure modes. The realistic risks for this stack are:

1. **Sanity outage or pricing change.** Content remains exportable as JSON, so the worst case is “migrate.”
2. **Cloudflare Pages outage.** Static assets can be re-pointed to any other static host (Netlify, GitHub Pages, S3+CloudFront) with a DNS change.
3. **Radio.co outage.** The live audio stream fails; the rest of the site is unaffected.
4. **Editor error.** Sanity history allows restoration of any document to any prior state.
5. **Bus factor.** The largest residual risk. Mitigated by this document, by inline code comments throughout the schema, and by the small surface area of the codebase itself.

§ 12 — COLOPHON

Colophon & Acknowledgements

This white paper documents work performed on the BearcastMedia.com website from April to May 2026 by the author. All code described is original work authored by Cam Garrison unless otherwise noted, with content authoring contributions from the rotating editorial staff of Bearcast Media.

Stack Summary

Frontend	Hand-authored HTML, CSS, vanilla JavaScript
Content layer	Sanity (project `3getuuyu`, dataset `production`)
Studio host	`bearcast.sanity.studio`
Site host	Cloudflare Pages
Audio	Radio.co (player widget + station-info JSON)
Typography	Playfair Display, Libre Baskerville, DM Sans, DM Mono (Google Fonts)
Icons	`@sanity/icons` (studio); emoji (public site, by design)

Author

Cam Garrison
GitHub: `@notChewy1324`
Web: `camgarrison.com`